



ChessAIThon

STRATEGIC GAMING, CODING, AND AI IN VET EDUCATION

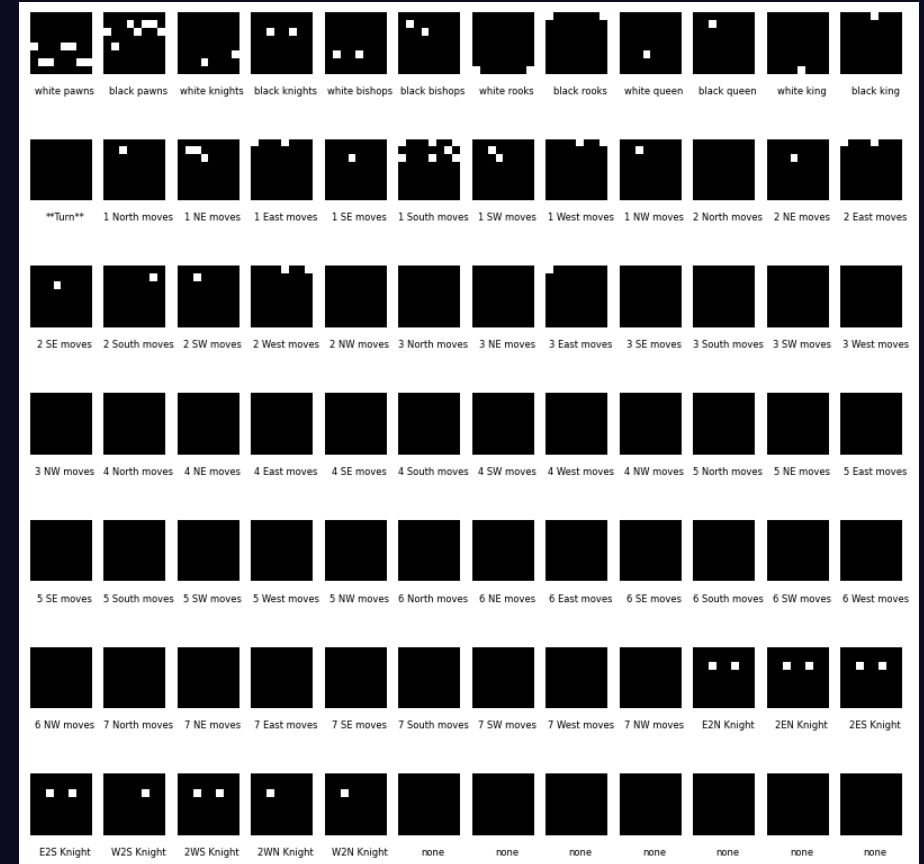
- How computers represent Chess boards and moves?
- How can I store boards and moves?
- How Chess AI works?
- How to train an AI for Chess?
- How to use AI to get the best move?
- How can I use my intelligence to improve AI?



Co-funded by
the European Union

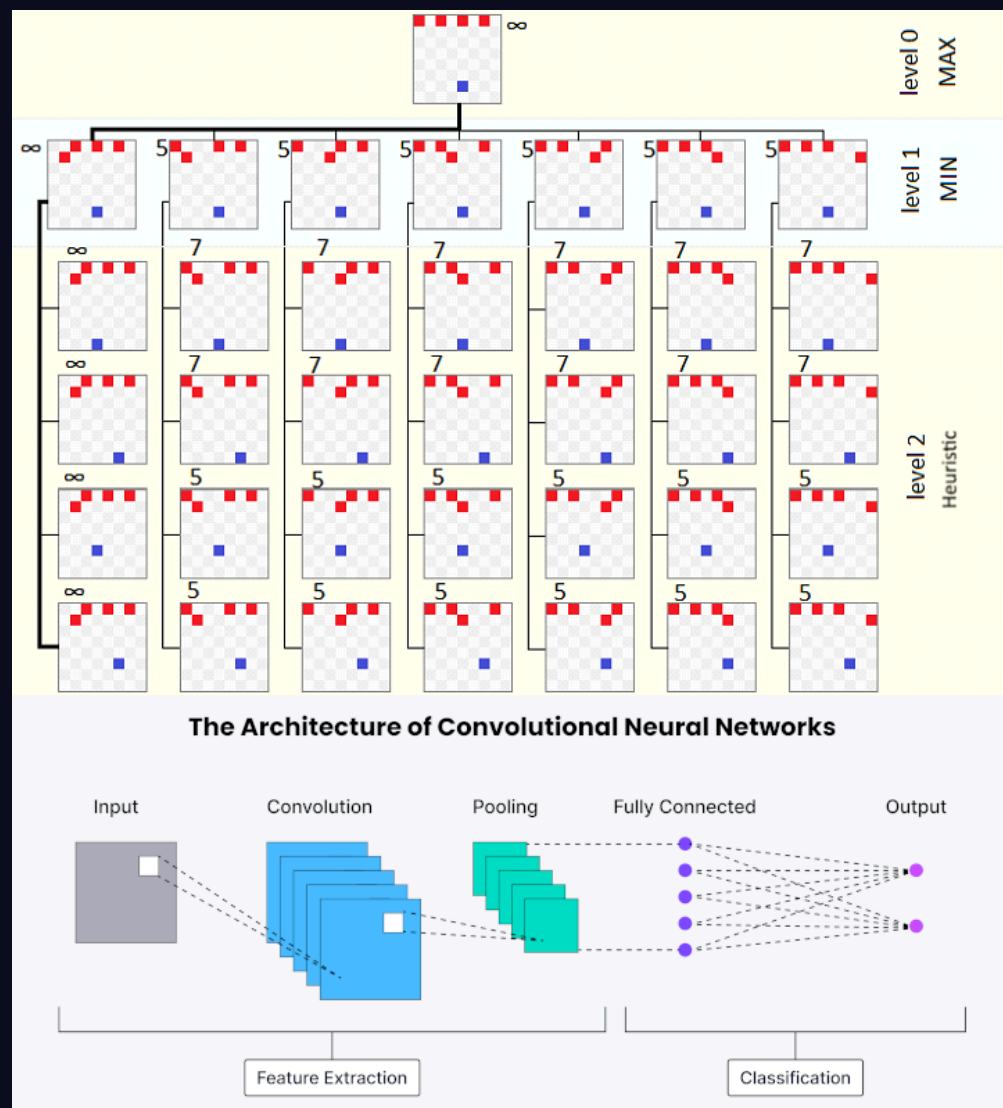
How computers represents Chess scenarios and moves?

- **Bitboards**: 64-bit integers representing squares. Highly efficient for bitwise operations.
- **Board Arrays**: Simple 8x8 grids for piece tracking.
- **Human Formats**: **FEN** (snapshot) and **UCI/SAN** (move notation).
- **AI Input (77x8x8)**:
 - 12 layers for piece positions.
 - 1 layer for turn indicator.
 - 64 layers for **legal move masks**.



How can I store scenarios and moves?

- **Standard Formats:** PGN (Games), FEN (Positions).
rnb1k1nr/pppp1ppp/5q2/2bP4/8/5N2/PPP1PPPP/RNBQKB1R w KQkq - 3 6
- **Data Science Stack:** CSV for simplicity, JSON for web APIs.
- **Parquet:** Columnar storage with **int64 compression**, reducing 10GB datasets to 1.5GB.
- **Version Control:** Using **Git/GitHub** to manage dataset iterations and collaborative student contributions.

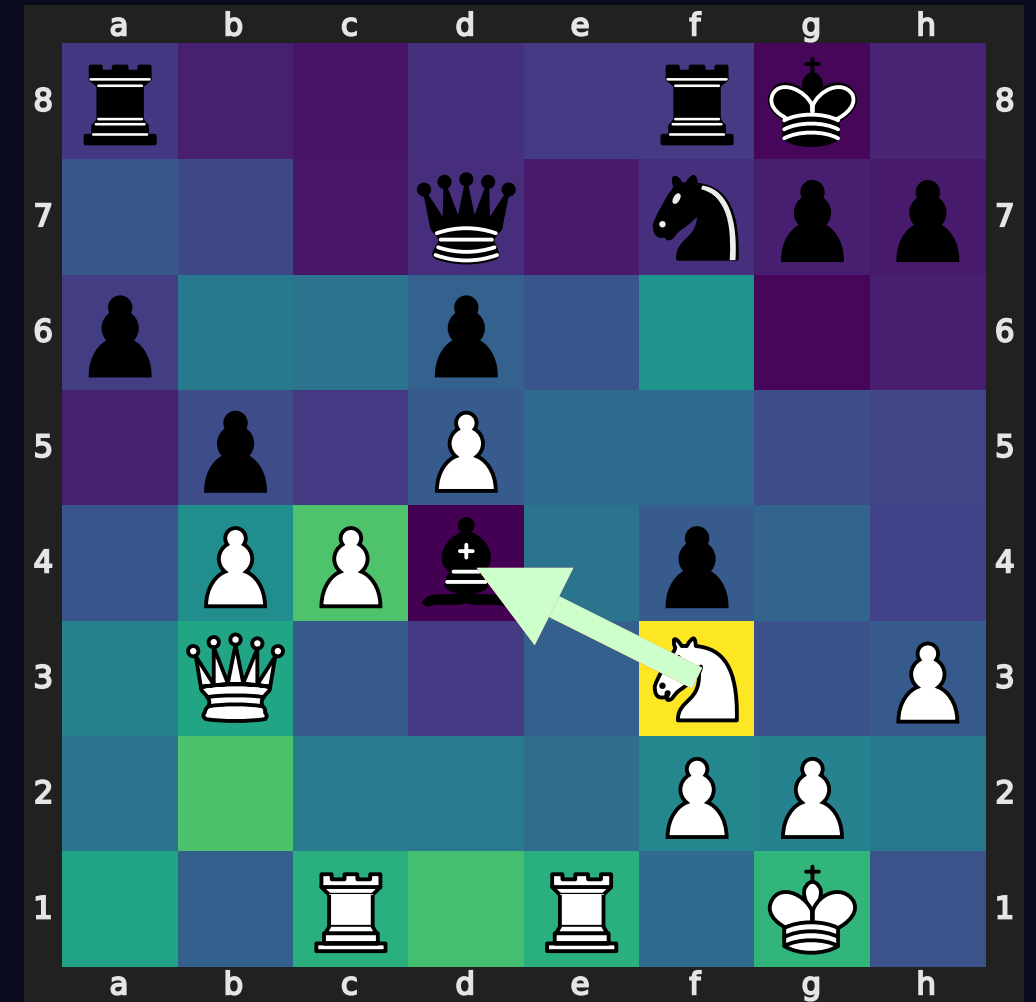


How Chess AI works?

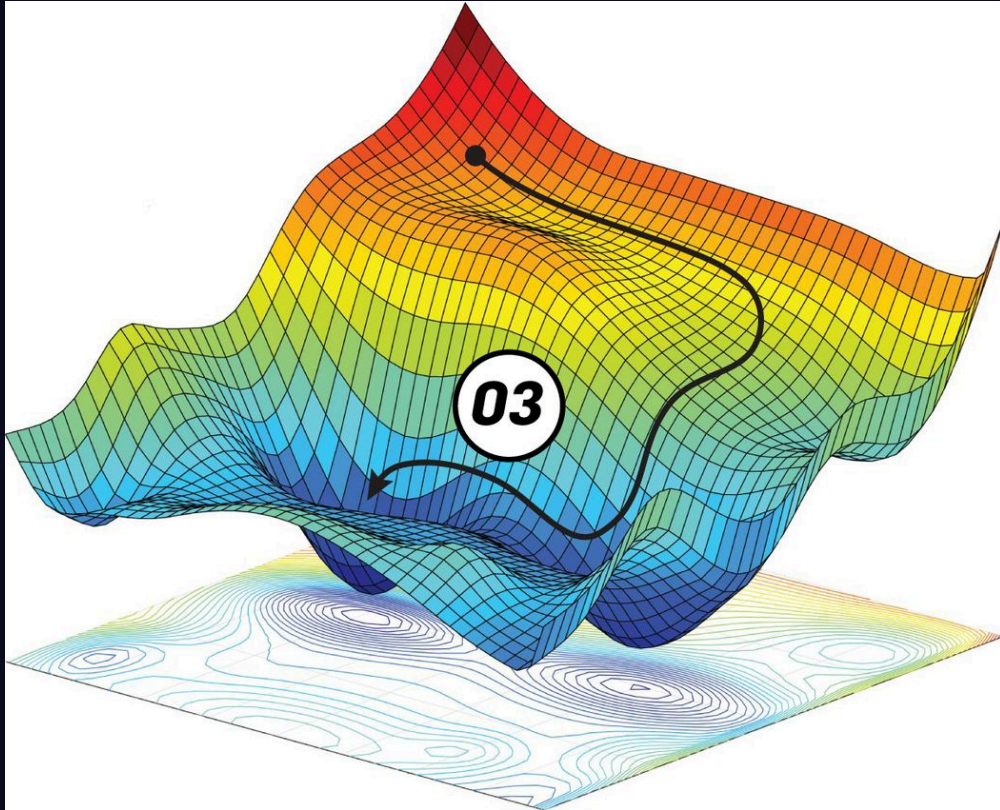
- **Rule-based Systems:** Brute-force calculation (Deep Blue).
- **Alpha-beta Search:** Heuristic evaluation (Stockfish).
- **Modern Neural Networks:** Intuition-based move prediction (AlphaZero).
- **Hybrid Model (ChessAITHon):**
 - **NN:** Provides "intuition" (Policy + Value).
 - **MCTS:** Provides "calculation" (Look-ahead search).

How to train an AI for Chess?

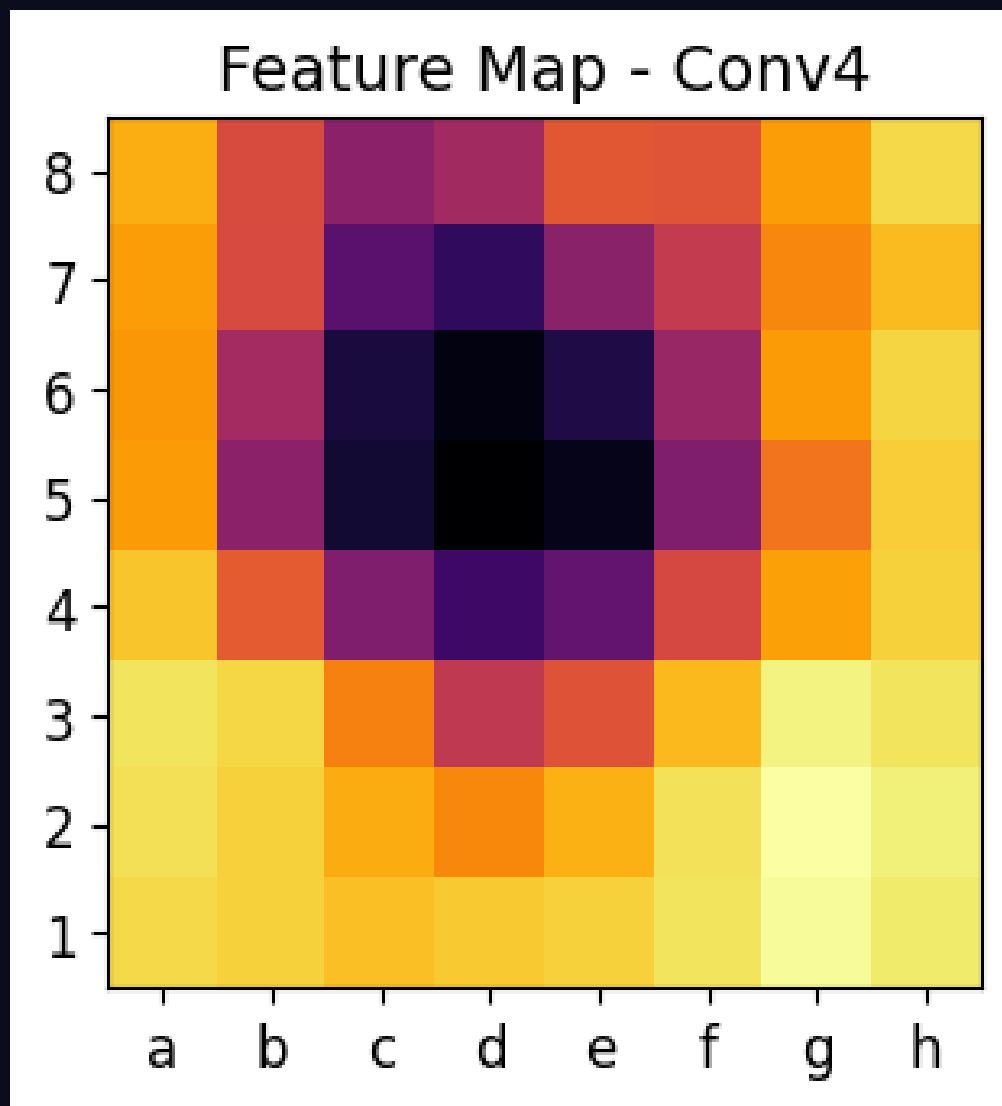
- **Our Choice:** Supervised learning from a curated dataset of human and simulated games.
- **Learning from Data:** Comparing NN predictions with the "best move" targets.
- **Human-Centric:** Trained on student moves to model human-like playstyles and complexity.



Deep Learning



- Each problem is a mathematical function with many parameters. We try parameters and check if returns the appropriate Y for the X
- **Loss and Gradient Descent** We try to minimize Loss and we try better parameters in the way to descent.
- **Learning Rate**: If we learn too fast we can miss the minimum.
- **Architecture**: We choose Convolutional Networks because chess has the same architecture as an image: 8x8 pixels of some color depth.

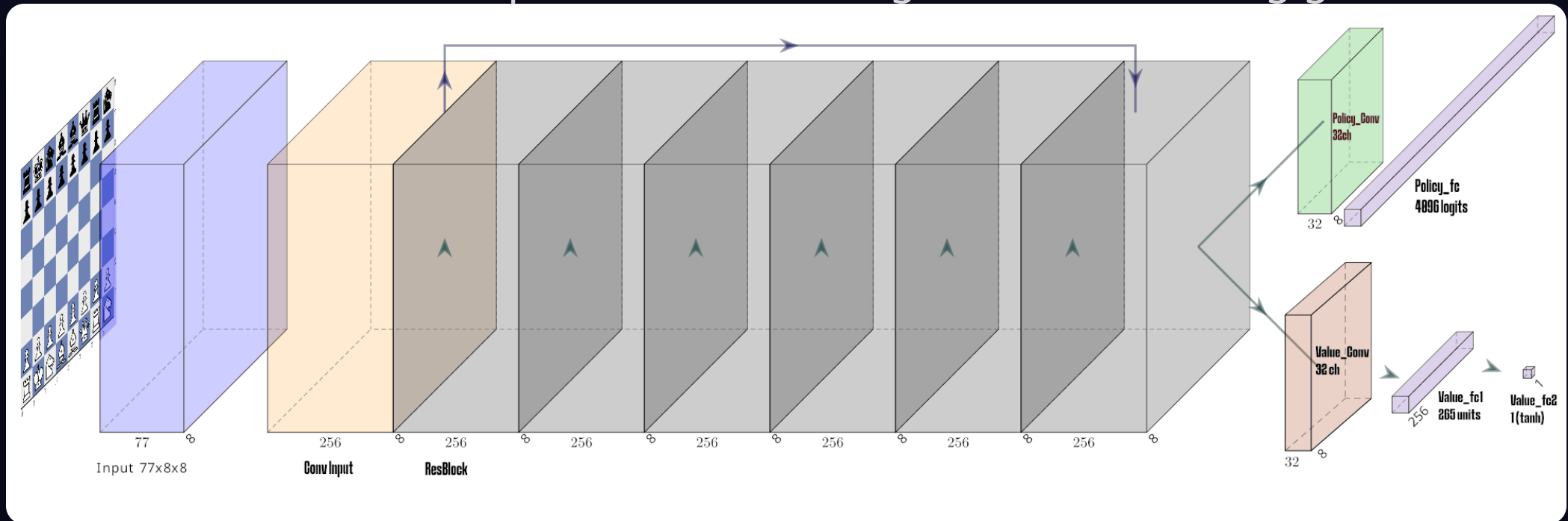


CNN

- **Bitboard representation:** 77x8x8
- **Chess is like an image:** 8x8 pixels of 77 bits of color depth.
- **Output:**
 - **Policies:** Probability distribution over 4096 possible moves.
 - **Value:** Predicting the win/loss outcome $[-1, 1]$.

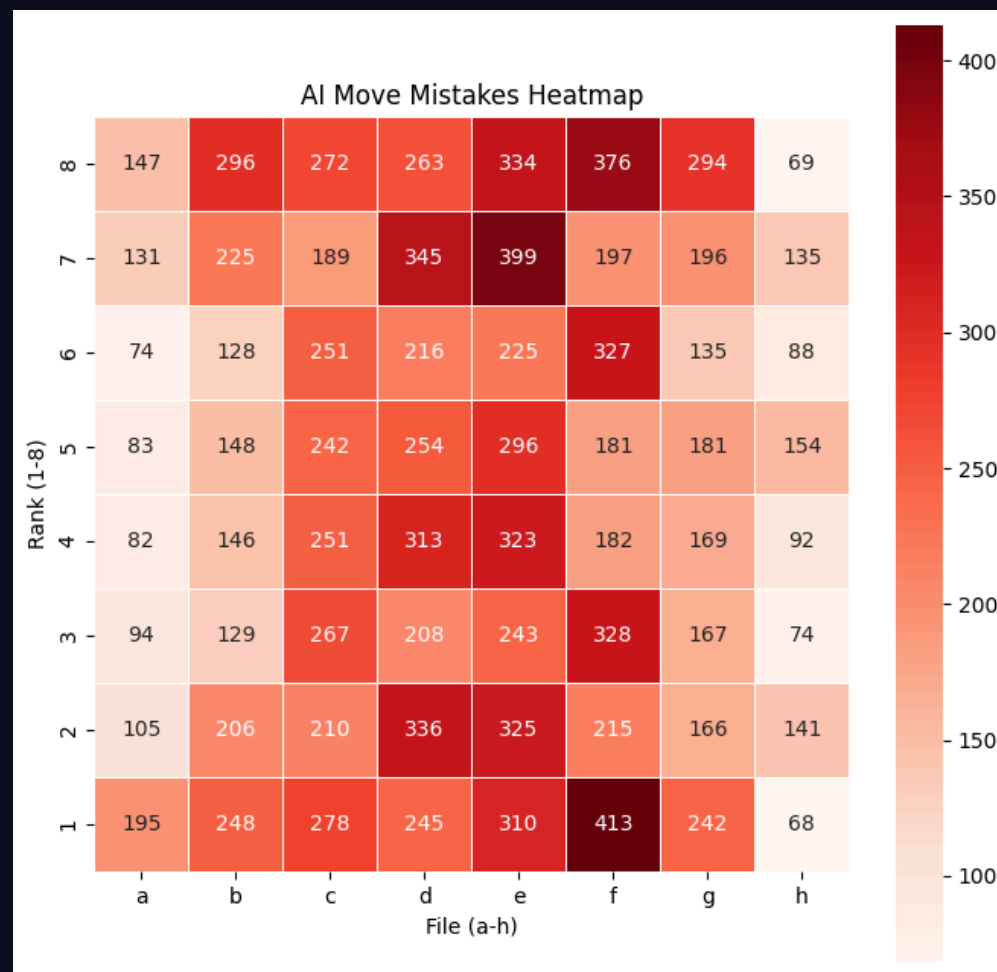
ChessNet

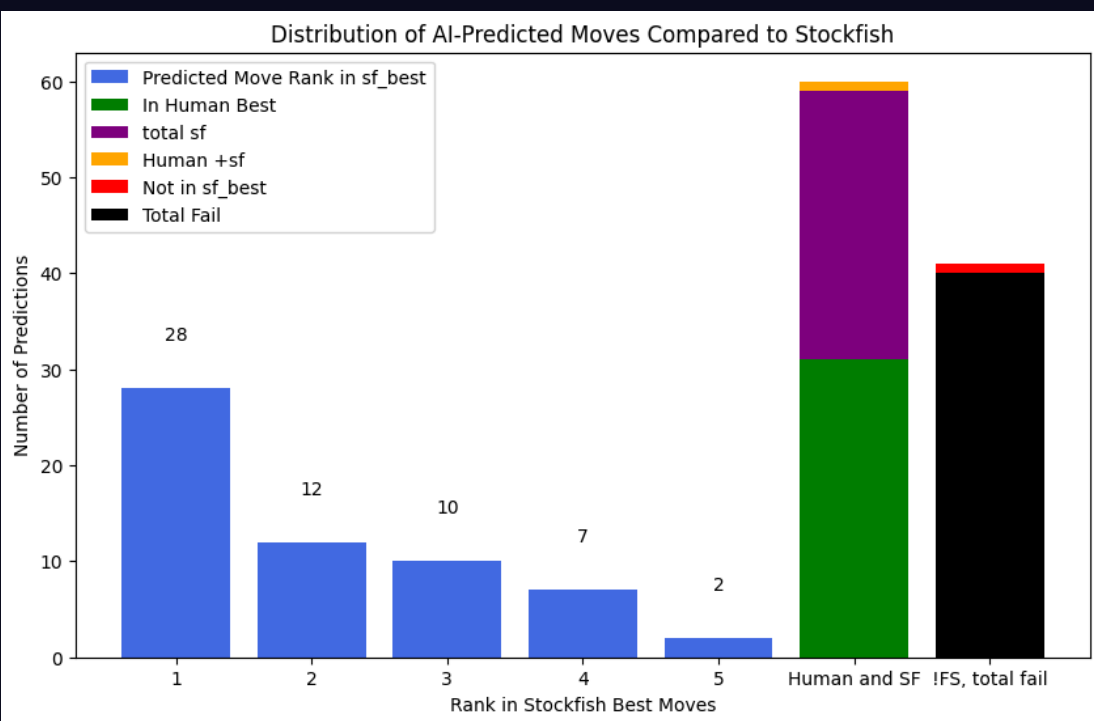
- **CNN layers:** For spatial feature extraction (piece patterns, tactical motifs).
- **Full Connected layers:** For integrating global board information and final predictions.
- **Residual Tower:** For deeper feature learning without vanishing gradients.



Dataset

- **Source:** Leela Chess Zero simulations (400 nodes) and Lichess puzzles.
- **Volume:** ~4,500,000 scenarios in Parquet format.
- **Diversity:** Includes balanced positions, tactical puzzles, and mate-specific scenarios.
- **Refinement:** Filtering out "decided" positions to prioritize informative learning signals ($0.1 < |\text{Value}| < 0.7$).



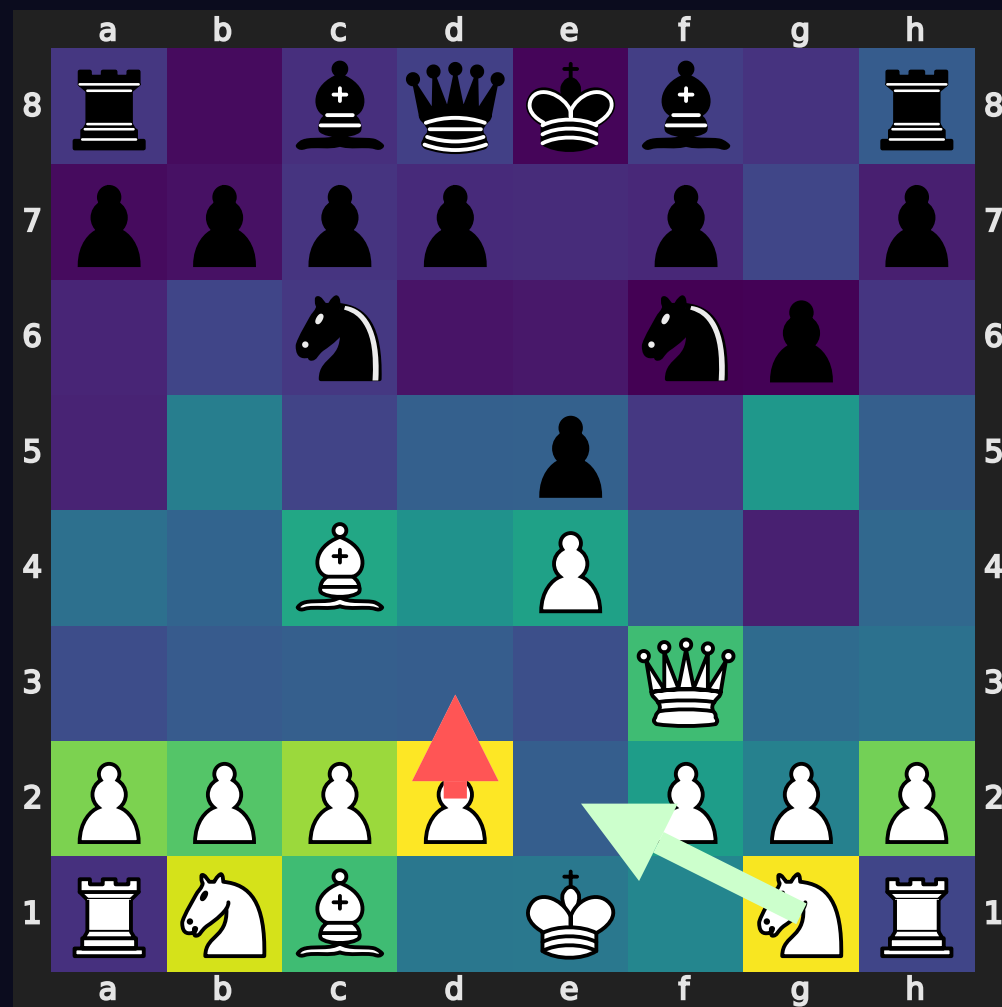


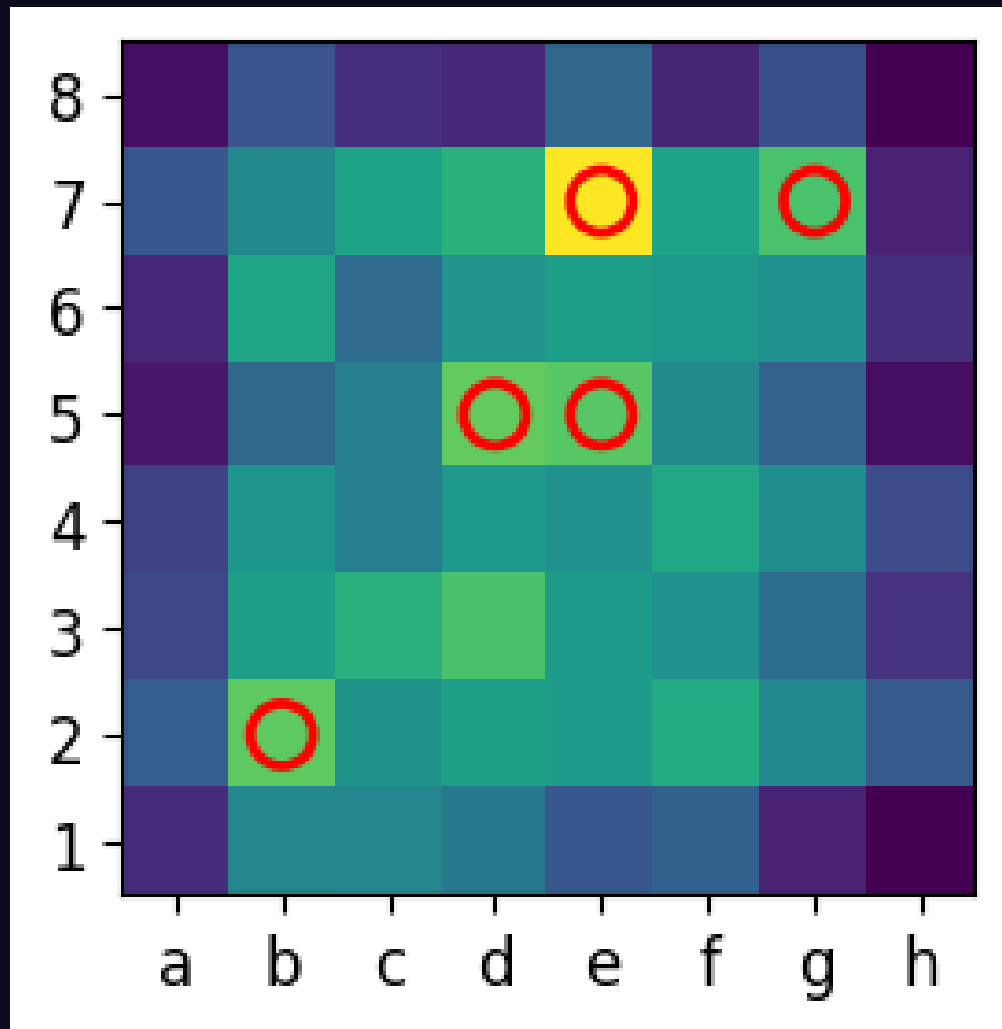
Results

- **Top-1 Accuracy: ~30%** (Matching Stockfish's perfect move).
- **Top-5 Accuracy: ~70%** (Sufficient to prune MCTS search space).
- **Success:** Even modest accuracy leads to strong strategic play when paired with MCTS.

How to get the best move?

- **MCTS Guided Search**: Simulation explores branches prioritized by the CNN Policy.
- **Exploration (PUCT)**: Balancing known good moves with less-explored alternatives.
- **Noise**: Dirichlet noise at the root node ensures variety and prevents deterministic traps.





How can I use my intelligence to improve AI?

- **ChessMinds Web App**: Play against other humans and contribute your best moves to the dataset.
- **Fine-tuning**: Use provided Notebooks to retrain the CNN on your personal games.
- **Community**: Modify architecture parameters or weights and share your version on HuggingFace.



Links

- **ChessMinds Web App:**
<https://chess-ai-thon.vercel.app/>
- **GitHub:**
<https://github.com/xxjcaxx/ChessAITHon>
- **HuggingFace Spaces Endpoint:**
<https://jocasal-chessAITHon.hf.space/predict>

Thank You

QUESTIONS & COLLABORATIVE AI DEVELOPMENT

GitHub: [xxjcaxx/ChessAIThon](https://github.com/xxjcaxx/ChessAIThon)